

“CampusConnect - Event Aggregator & Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Tejaswin Amara

2520090104

Namish Aditya Vallala

2520090110

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled ***“CampusConnect - Event Aggregator & Management System”*** is a record of bonafide work of *Tejaswin Amara – 2520090104, Namish Aditya Vallala - 2520090110* submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

Tejaswin Amara – 2520090104

Namish Aditya Vallala – 2520090110

KL (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled ***"CampusConnect - Event Aggregator & Management System"*** is a bonafide work done and submitted by *Tejaswin Amara – 2520090104, Namish Aditya Vallala - 2520090110* in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT** (25CS1302E), Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the **Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Tejaswin Amara – 2520090104

Namish Aditya Vallala – 2520090110

ABSTRACT

CampusConnect is a full-stack campus event aggregation and student registration platform designed to centralize discovery, controlled event administration, organizer workflows, and engagement telemetry for a university environment. The system replaces fragmented event discovery across clubs, departments, social channels, and external forms with a single web application backed by a relational database. The core runtime uses Spring Boot, Spring Security, Spring Data JPA, Hibernate, MySQL, and Flyway, while the modern frontend is implemented with React, Vite, TypeScript, Tailwind CSS, React Query, and form-validation tooling. Legacy Thymeleaf templates are retained for selected server-rendered flows.

The platform supports event search and category filtering, event detail views, role-based administration, club-scoped organizer workspaces, event creation and lifecycle management, external registration redirection, attendance check-in, CSV export, explainable event recommendations, and real-time Server-Sent Events (SSE) for capacity and organizer check-in telemetry. Database integrity is enforced through primary keys, foreign keys, unique constraints, CHECK constraints, indexes, and Flyway migrations. Registration workflows use a database uniqueness invariant on (user_id, event_id), while service-level pessimistic locking is used for concurrency-sensitive operations.

The repository also contains evidence for database engineering concepts such as normalization, query optimization, concurrency stress testing, transactional outbox design, and operational observability. The latest main-branch GitHub Actions run reviewed for this report completed successfully across frontend verification, backend verification, and security scanning. At the same time, the repository contains some transition-state inconsistencies - notably between the documented runtime versions and the current Maven/CI configuration, and between the standalone schema.sql description and the later V5 multi-role/club migration. These have been recorded as limitations rather than silently reconciled. Overall, CampusConnect demonstrates a layered modular-monolith architecture with a strong academic emphasis on relational integrity, secure access control, testability, and deployment reproducibility.

INDEX

The report follows the same academic organization as the reference documentation: introductory material, system requirements, technology stack, architecture, design, implementation, features, testing, deployment, challenges and limitations, future enhancements, conclusion, references, and appendices. The page numbers below are updated after layout rendering.

S.No.	Chapter	Topics	Page
Chapter 1	Introduction	1.1 Background of the Project; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users / Audience	8
Chapter 2	System Requirements	2.1 Hardware Requirements; 2.2 Software Requirements; 2.3 and Development Tools Frameworks; 2.4 Development Environment Contract	10
Chapter 3	Technology Stack	3.1 Frontend; 3.2 Backend; 3.3 Database; 3.4 Version Control and CI/CD; 3.5 Authentication, Security and Observability	12
Chapter 4	System Architecture	4.1 High-Level Architecture Diagram; 4.2 Layer Description; 4.3 Deployment Architecture	13
Chapter 5	Design	5.1 Database Design Principles; 5.2 ER Diagram / Schema; 5.3 and Design Decisions Assumptions	15
Chapter 6	Implementation	6.1 Module-wise Implementation; 6.2 Front-end Logic; 6.3 API Integration; 6.4 Authentication and Authorization	17
Chapter 7	Features	7.2 Student Features; Administrator Features; 7.3 Organizer Features; 7.4 Recommendations, Telemetry and Check-in	18
Chapter 8	Testing	8.1 CI Pipeline Testing; 8.2 Backend Tests; 8.3 Frontend Tests; 8.4 Concurrency, Security and Performance; 8.5 Tools; 8.6 Results	20
Chapter 9	Deployment	9.1 Local Deployment; 9.2 Environment Configuration; 9.3 Container Deployment; 9.4 Firebase Hosting / Hybrid Delivery	22

S.No.	Chapter	Topics	Page
Chapter 10	Challenges & Limitations	10.1 Version Drift; 10.2 Schema/Migration Drift; 10.3 In-memory Rate Limiting; 10.4 External Registration Model; 10.5 Hybrid Frontend	23
Chapter 11	Future Enhancements	11.1 Planned Features; 11.2 Integrations and Optimizations	24
Chapter 12	Conclusion	12.2 Summary; Achievements; 12.3 Skills Learned	25
Chapter 13	References	Documentation, tools and resources used	26
Chapter 14	Appendices	A Sample Code; B Setup; C Demo Credentials Note; D API Summary; E Visuals; F Verification Snapshot	27

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

University event information is commonly distributed across student clubs, departmental notices, social platforms, messaging groups, and external registration forms. This fragmentation makes it difficult for students to discover events consistently and makes it harder for administrators to maintain a reliable view of event lifecycle, registrations, capacity, and participation. CampusConnect was designed as a centralized campus event catalogue and management platform that brings these workflows into a single database-backed system.

The repository description positions CampusConnect as a full-stack event management middleware for universities. Its implemented scope extends beyond a basic CRUD catalogue: the current codebase includes role-based access control, organizer club workspaces, event recommendations, attendance check-in, real-time SSE telemetry, transactional event publishing records, observability endpoints, and automated verification through GitHub Actions.

1.2 Problem Statement

The primary problem is the absence of a single, controlled source of truth for campus events and their associated user interest records. Students need to search and filter events, inspect event details, and follow an authoritative registration path. Administrators need privileged event management, analytics, and export capabilities. Club organizers need scoped access to their own events and attendee information. The system must perform these operations without allowing unauthorized mutation or duplicate user-event registration records.

1.3 Scope of the Project

- Centralized event discovery with search, filtering, pagination, historical/ongoing views, and detailed event pages.
- Administrator workflows for event creation, editing, deletion, analytics, roster inspection, image handling, filtering, and CSV export.
- Organizer workflows scoped to a club, including event creation, attendee inspection, and attendance check-in.
- Student/guest access to public event discovery and event registration-interest flows with an external registration redirect.
- Database-backed integrity using constraints, indexes, Flyway migrations, and concurrency-safe service logic.
- Real-time capacity and check-in telemetry using Server-Sent Events (SSE).
- Containerized deployment and reproducible automated verification through Docker and GitHub Actions.

Scope boundary from the repository: the registrations table records interest/registration intent before external registration redirection; it is not an authoritative internal payment or seat-allocation ledger.

1.4 Objectives

1. Provide a centralized and searchable catalogue of campus events.
2. Enforce role-based access so administrative and organizer operations are protected.
3. Maintain relational integrity through keys, foreign keys, unique constraints, CHECK constraints, and controlled migrations.
4. Prevent duplicate user-event registration records even under concurrent requests.
5. Provide explainable recommendations using relational event and interest data.

6. Support real-time capacity and organizer check-in updates without requiring page refreshes.
7. Provide a reproducible build/test/deployment workflow suitable for academic evaluation and future extension.
8. Demonstrate database engineering concepts including normalization, indexing, query optimization, transactions, and distributed-ready outbox design.

1.5 Importance of the Application

CampusConnect improves information visibility by making events discoverable through a consistent interface and improves operational control by separating student, organizer, and administrator capabilities. From a database engineering perspective, it also demonstrates how integrity constraints and transaction boundaries can be used to make business rules enforceable rather than merely advisory. The repository therefore serves both as a usable campus application and as evidence for the course concepts required by 25CS1302E.

1.6 Target Users / Audience

Actor	Primary Responsibilities	Access Boundary
Student / Guest	Browse, search, filter, view event details, receive recommendations, follow external registration path	Public/event endpoints; no privileged admin mutations
Organizer	Manage club-scoped events, inspect attendees, perform check-in, monitor live check-in telemetry	ROLE_ORGANIZER or ROLE_ADMIN; event ownership enforced by service checks
Administrator	Manage the platform, events, analytics, exports, and operational controls	ROLE_ADMIN on `/admin/**` and `/api/admin/**`
Database / Release Operator	Provision MySQL, run migrations, configure secrets, inspect health and CI results	Infrastructure and repository responsibility

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The repository is designed to run as a conventional Java web application with MySQL. For a local demonstration, a modern dual- or quad-core development machine with 8 GB RAM is a practical baseline, especially when Docker, MySQL, a Java build, Node tooling, and browser-based testing are used together. The repository does not define a formal minimum hardware specification, so this requirement is an operational recommendation rather than an implementation constraint.

Resource	Recommended Local Baseline	Purpose
Processor	2-4 CPU cores or better	Java build, MySQL, frontend tooling, browser tests
Memory	8 GB RAM or better	Docker + JDK + Node/pnpm + browser test execution
Storage	5 GB free	Maven/Node caches, Docker layers, project sources, uploads
Network	Internet access for dependency/image retrieval	Maven/ npm/pnpm packages and container images

2.2 Software Requirements

- JDK 21 for the current Maven/CI build contract.
- Maven wrapper (``mvnw` / `mvnw.cmd``) for backend build and verification.
- Node.js 20 and pnpm 9 for the React/Vite frontend CI path.
- MySQL 8.x; the current CI service uses MySQL 8.0 while repository documentation also references MySQL 8.4.
- Git and a modern web browser.
- Docker and Docker Compose for containerized local deployment.
- Optional development tools: Visual Studio Code, IntelliJ IDEA, MySQL Workbench, Postman or equivalent HTTP tooling.

2.3 Development Tools and Frameworks

Area	Tools / Frameworks	Role in Project
Backend	Spring Boot 4.1.0, Spring MVC, Spring Security, JPA, Hibernate	HTTP endpoints, services, security, persistence
Databases	MySQL, InnoDB, Flyway	Transactional storage, constraints, migrations
e Frontend	React 19, Vite, TypeScript, Tailwind, React Query, RHF, Zod	SPA UI, server state, forms, validation
Testing	JUnit, Mockito, Vitest, Testing Library, Playwright, Axe	Unit, integration, component, E2E and accessibility checks
Operations	Actuator, Micrometer/Prometheus, Logback, Caffeine	Health, metrics, structured logging, caching
Security	BCrypt, Spring Security, Bucket4j	Credential hashing, authorization, login rate limiting

2.4 Development Environment Contract

For reproducible local and CI execution, the current build contract is JDK 21 for Maven/Spring Boot, Node.js 20 with pnpm 9 for the React frontend, and MySQL 8.x for the relational database. The repository is designed so that these toolchains can be run directly from source or reproduced through the Docker build and Compose stack.

Component	Current Development Value	Evidence in Repository
Backend	JDK 21; Spring application port 8080 by default	pom.xml, CI workflow, application.properties
Frontend	Node 20; pnpm 9; Vite dev server base URL 5173	CI workflow, package.json, Playwright configuration
Database	MySQL 8.0 service in CI; MySQL 8.x local	GitHub Actions workflow, Docker Compose
Operations	target Docker Compose health-gated startup	docker-compose.yml and Dockerfile

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The modern frontend is located under `frontend-redesign/` and is a React 19 application built with Vite and TypeScript. The dependency set includes Tailwind CSS, React Query for server state, React Hook Form and Zod for forms and validation, Motion for UI transitions, and Lucide icons. Vite is configured to split major vendor packages into separate chunks. The frontend CI gate runs formatting/lint checks through Biome, strict TypeScript compilation, Vitest tests with coverage, Playwright browser tests, and Axe accessibility checks before building production assets.

The repository also retains a set of Thymeleaf templates under `src/main/resources/templates`. This indicates an intentional hybrid/transition architecture rather than a purely client-rendered system.

3.2 Backend

The backend is a modular Spring MVC application in the `com.tejaswin.campus` package. Controllers expose browser and JSON endpoints; services encapsulate event lifecycle, registration, recommendations, sessions, security checks, and telemetry; repositories provide JPA access. `EventService` is the primary business service, while dedicated controllers expose administrator, organizer, authentication, event, and telemetry behavior.

3.3 Database

The core relational store is MySQL using the InnoDB storage engine. The documented master SQL schema contains `users`, `events`, `registrations`, and `outbox_events`. Flyway migrations V1 through V5 extend this structure, with V5 adding a `clubs` table and multi-role/club/check-in fields. The schema uses primary keys, foreign keys, unique constraints, CHECK constraints, and secondary B-tree indexes. Registration uniqueness is enforced by the composite key `(user_id, event_id)`.

3.4 Version Control and CI/CD

Git and GitHub are used as the source-control system. The current workflow file is named `Production Delivery Pipeline`, but its present jobs are verification and security gates rather than an end-to-end production deployment. The frontend gate uses Node 20 and pnpm 9; the backend gate uses JDK 21 and a MySQL 8.0 service; the security gate runs Gitleaks and Trivy. A separate Firebase Hosting configuration is present in the repository for the frontend build.

3.5 Authentication, Security and Observability

- BCrypt password hashing with configurable strength (default 12 in application configuration).
- Spring Security role boundaries for student, organizer, and administrator workflows.
- Session fixation protection through Spring Security session migration.
- CSRF protection for browser flows, with API paths explicitly configured for JSON clients.
- Bucket4j rate limiting for POST `/admin/login`, backed by a Caffeine in-memory bucket cache.
- Security audit logging and structured error handling without stack-trace exposure to clients.
- Spring Boot Actuator health and Prometheus-compatible metrics endpoints.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

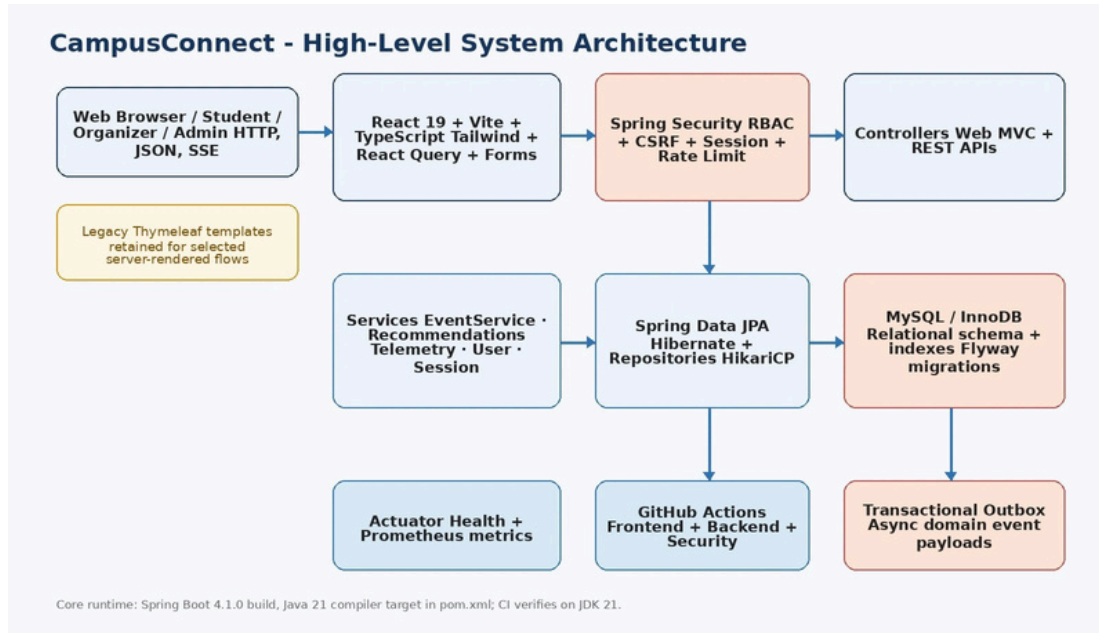


Figure 4.1 - CampusConnect high-level architecture

Source: repository structure, `pom.xml`, `Dockerfile`, `SecurityConfig.java`, service/repository packages, and CI workflow.

4.2 Description of Each Layer

Layer	Responsibilities	Key Components
Presentation	Renders the student, organizer, and admin user experience; sends API requests and listens for SSE updates	React/Vite frontend; retained Thymeleaf templates; Web browser
Security / Edge	Authentication, session management, authorization, CSRF, rate limiting and error boundaries	Spring Security, CustomUserDetails, RateLimitingFilter
Application	Event lifecycle, registration workflows, recommendations, organizer scope, check-in and telemetry	EventService, RecommendationService, EventTelemetryService, UserService
Persistence	ORM mapping, repositories, lock queries, data access and connection pooling	JPA entities, repositories, Hibernate, HikariCP
Data / Operations	Relational persistence, schema migration, asynchronous event records, health and metrics	MySQL/InnoDB, Flyway, outbox_events, Actuator/Prometheus

4.3 Deployment Architecture

The Dockerfile builds the React frontend with Node 20 and pnpm, packages it into the Spring Boot static resources, then produces a non-root Java 21 runtime image. Docker Compose provisions a MySQL service and the application service, waits for database health before starting the application, and persists MySQL data and uploaded media in named volumes. The application port is configurable through the `PORT` environment variable and defaults to 8080 in the current configuration.

The repository also contains Firebase Hosting configuration targeting the `frontend-redesign/dist` output with an SPA rewrite to `index.html`. This should be regarded as a frontend hosting/integration path; MySQL remains the authoritative application database in the main Spring runtime.

CHAPTER 5: DESIGN

5.1 Database Design Principles

- Entity separation: authentication identity, event metadata, registration associations, clubs, and asynchronous outbox records are stored independently.
- Referential integrity: foreign keys connect registration records to users/events and V5 adds club references.
- Uniqueness: composite `(user\_id, event\_id)` constraint prevents duplicate user-event records.
- Domain validation: CHECK constraints restrict event status, registration status, positive capacity, and valid date ordering.
- Query-aware indexing: date, category/date, status/date, attendee/status, student/status, and outbox polling indexes are defined.
- Migration control: Flyway provides ordered schema evolution instead of relying on unmanaged DDL drift.

5.2 ER Diagram / Database Schema

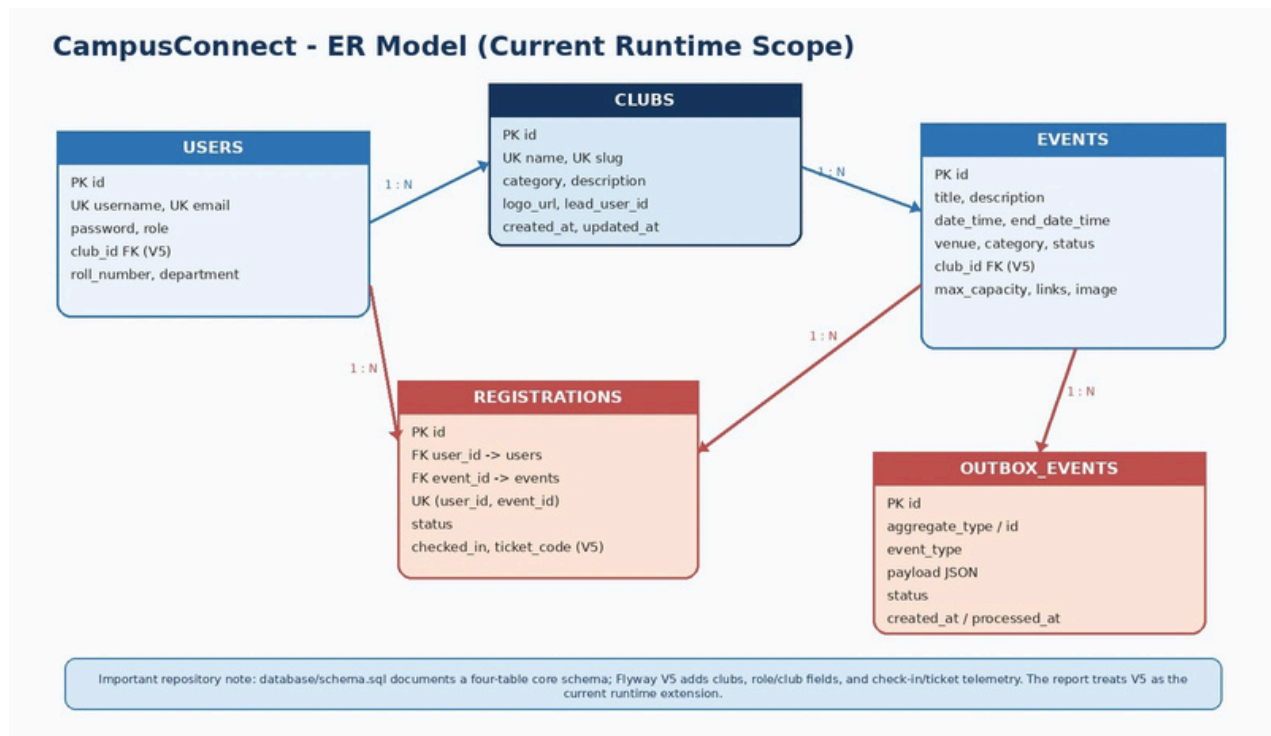


Figure 5.1 - Current conceptual ER model including the V5 club and check-in extension

The repository's `database/er-diagram.md` focuses on the four core tables; the figure above extends that model to reflect the V5 migration currently present on main.

Entity	Purpose	ImportantConstraints / Relationships
USERS	Authentication and user identity	Unique username/email; role; optional club association
CLUBS	Organizer organization boundary	Unique name/slug; one club may own many users/events
EVENTS	Campus event catalogue	Lifecycle status, scheduling, capacity, category, media, optional club

Entity	Purpose	Important Constraints / Relationships
REGISTRATIONS	User-event interest/registration association	Unique user-event pair; status; V5 check-in/ticket fields
OUTBOX_EVENTS	Transactional domain-event queue record	JSON; PENDING/PROCESSED/FAILED lifecycle; indexed for polling

5.3 Design Decisions and Assumptions

- Registration means application-level interest/registration intent; actual authoritative ticketing or payment may remain external.
- Recommendation ranking is deterministic and server-side. Client-supplied scores or external recommendation documents are not trusted.
- Event-specific organizer actions are constrained through `SecurityService` checks rather than role membership alone.
- SSE is used for live telemetry because capacity and check-in updates are naturally push-oriented.
- The transactional outbox table is implemented as a database primitive, while a broader distributed messaging architecture remains a documented evolution path.
- The repository is a modular monolith. Splitting services or adding Kafka/Redis is treated as an evolution choice, not an assumed current deployment.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

Module	Primary Classes / Paths	Implemented Responsibilities
Authentication	AuthController, UserService, CustomUserDetailsService	Login, registration, user lookup, password handling, session bridge
Events	EventController, EventApiController, EventService, EventRepository	Catalogue, search/filter, CRUD, validation, pagination, event state
Admin	AdminController	Administrative dashboard, lifecycle management, analytics, media upload, export
Organizer	OrganizerController, ClubRepository, SecurityService	Club-scoped event workspace, attendees, check-in, access validation
Registration	Registration model/repository + EventService	User-event association, uniqueness, interest state, check-in data
Recommendations	RecommendationService, RecommendedEvent	Category-affinity + popularity heuristic with explainable reasons
Telemetry	EventTelemetryController, EventTelemetryService	SSE capacity stream and organizer check-in stream
Data migration	Flyway V1-V5	Ordered schema creation, indexes, integrity checks, outbox, multi-role/club extension

6.2 Front-end Logic

The React frontend uses Vite for development and production bundling and organizes server state around React Query. Forms are validated with React Hook Form and Zod. The application consumes the Spring JSON API and SSE streams rather than connecting directly to MySQL. The build is code-split using Vite manual chunks for React, motion, query, icons, and forms. The retained Thymeleaf templates support legacy server-rendered flows that coexist with the React SPA delivery model.

6.3 API Integration

The backend exposes REST endpoints under `/api/...` and also serves web-facing routes. `EventApiController` provides event listing, details, and registration flows, while `OrganizerController` provides club-scoped organizer capabilities. `EventTelemetryController` exposes `text/event-stream` endpoints for capacity and live check-in updates. OpenAPI documents are generated through springdoc and are available at `/v3/api-docs` with Swagger UI under `/swagger-ui.html`.

6.4 Authentication and Authorization

Authorization is configured in `SecurityConfig.java`. Public event discovery and authentication endpoints are explicitly permitted; `/admin/\*\*` and `/api/admin/\*\*` require `ROLE\_ADMIN`; `/api/organizer/\*\*` requires `ROLE\_ORGANIZER` or `ROLE\_ADMIN`. Event-specific organizer operations additionally use the application-level expression `@securityService.canManageEvent(principal, #id)`. Session fixation protection, CSRF handling, CSP/referrer headers, and login rate limiting are also configured in the security layer.

CHAPTER 7: FEATURES

7.1 Student Features

- Browse upcoming and historical events.
- Search by event title or venue and filter by category.
- Open event detail pages containing schedule, venue, capacity, registration links, and media where available.
- Record interest/registration intent and follow an external registration link.
- Receive explainable upcoming-event recommendations derived from prior interest/attendance and current event popularity.
- Observe live event capacity telemetry through SSE without full-page refresh.

7.2 Administrator Features

- Create, edit, publish, cancel, complete, and delete events according to lifecycle rules.
- Upload and manage event images.
- View operational analytics and registration information.
- Export event/registration records to CSV.
- Inspect event rosters and administrative views under protected routes.
- Use secure login protected by session security, BCrypt, and login rate limiting.

7.3 Organizer Features

- Access a club-scoped organizer workspace.
- Create and list events associated with the organizer's club.
- View event attendee rosters with registration metadata.
- Check in attendees using ticket code or roll number.
- Observe live check-in telemetry through the organizer SSE stream.
- View club-level totals for events, RSVPs, and checked-in attendees.

7.4 Recommendations, Telemetry and Check-in

Feature	Implementation	Behavior
Recommendations	RecommendationService	Category affinity from registrations/attendance plus returns scored, explainable results and excludes already registered events
Capacity telemetry	EventTelemetryService + SSE	Broadcasts current registered count, max capacity, remaining seats and saturation percentage after registration commits
Check-in	OrganizerController + Registration fields	Updates checked_in/check_in_time and emits a live check-in event to organizer subscribers
Transactional outbox	outbox_events	Stores event payloads and processing state distribution; for reliable asynchronous broader message bus deployment remains future architecture

7.5 Feature Verification Evidence

Feature	Implementation Evidence	Verification Path
Event discovery	EventController / EventRepository search, category and pagination queries	Frontend browse/search flow; repository tests
Admin boundary	SecurityConfig + ADMIN route rules	Security controller tests and protected route checks
Organizer scope	OrganizerController + SecurityService.canManageEvent	Organizer tests and event-specific access checks
Duplicate registration prevention	uk_user_event + pessimistic event lock	EventServiceConcurrencyTest and database constraint
Live telemetry	EventTelemetryService + SSE	Telemetry controller tests and post-commit broadcast path

CHAPTER 8: TESTING

8.1 CI Pipeline Testing

The latest main-branch GitHub Actions run reviewed for this report was run 87 (commit `b51fd90e8321c432d3f654945aa2dfef3a60aeb1``) on 29 September 2026. The three verification/security jobs all completed successfully: Frontend Verification, Backend Verification, and Security & Secret Scan.

Gate	Checks	Observed Result
Frontend Verification	pnpm install; Biome check; TypeScript compile; Vitest coverage run; Playwright + Axe; production build	Job completed successfully
Backend Verification	JDK 21; MySQL service; Maven <code>verify`</code>	Job completed successfully
Security & Secret Scan	Gitleaks; Trivy filesystem scan	Job completed successfully; Trivy is configured non-blocking in workflow

8.2 Backend Unit and Integration Testing

The main branch contains 21 Java test classes under `src/test/java/``, covering configuration, controllers, security filters, models, services, concurrency, session behavior, and integration-style controller tests. The test suite includes a dedicated `EventServiceConcurrencyTest``, security tests for admin/organizer boundaries, recommendation service tests, and telemetry-related controller tests.

8.3 Frontend Component, E2E and Accessibility Testing

The frontend tree contains 89 TypeScript/TSX source files and 19 test/spec files. The CI workflow runs Vitest with coverage, then launches Playwright against Chromium and exercises Axe accessibility checks before the production build is produced. This is stronger than manual browser verification alone because the pipeline performs repeatable automated checks on the client bundle.

8.4 Concurrency, Security and Performance Testing

The repository documentation records a 10-thread duplicate-registration stress test in which one registration succeeds and nine are rejected by the unique user-event constraint, as well as a distinct-user test in which eight simultaneous registrations succeed without deadlocks. The implementation uses pessimistic event-row locking through `findByIdForUpdate`` plus the database unique constraint on `(user_id, event_id)``.

Query optimization documentation records a before/after experiment for category-filtered event queries: a full scan and filesort are replaced by the composite `(category, date_time)`` B-tree index, with a documented cost reduction from 1.45 to 0.70 and filesort elimination. Because this benchmark is repository documentation rather than a fresh measurement performed during this report generation, it is reported as documented evidence rather than re-measured evidence.

The security pipeline also runs Gitleaks and Trivy. The workflow intentionally sets the Trivy step to continue and uses exit code 0, so completion of that job should not be interpreted as a guarantee of zero vulnerabilities.

8.5 Tools Used for Testing

- JUnit 5 / Spring Boot Test / Mockito
- Vitest / Testing Library

- Playwright / Axe
- GitHub Actions
- MySQL test service in CI
- Gitleaks and Trivy

8.6 Test Results

Test / Verification Area	Expected Outcome	Repository Evidence
Backend build verification	Maven verify completes	Latest CI backend gate: success
Frontend type/build verification	TypeScript compiles and production build succeeds	Latest CI frontend gate: success
Duplicate registration race	No duplicate user-event records	Concurrency test documentation: 1 success / 9 duplicates rejected
Role boundary	Unauthorized access rejected	Security controller/filter tests and Spring Security configuration
Accessibility regression	Playwright + Axe checks complete	Latest CI frontend gate includes both steps
Secret/vulnerability scanning	Security scans complete under configured policy	Latest CI security gate: success; Trivy non-blocking

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy Locally

1. Copy `.env.example` to `.env` and replace placeholder credentials with local values.
2. Start MySQL and the application through Docker Compose using `docker compose up --build -d`, or run MySQL separately.
3. For a source-based run, ensure JDK 21 and MySQL are available, then use `./mvnw -B verify` for verification and `./mvnw spring-boot:run` for local execution.
4. The modern frontend can be run independently from `frontend-redesign/` using pnpm, or packaged into the Spring Boot static resources by the Docker build.
5. Open the application at the configured `PORT` (8080 by default) and verify `/actuator/health` and `/v3/api-docs`.

9.2 Environment Configuration

Variable Group	Examples	Purpose
Server	PORT, APP_BIND_ADDRESS	HTTP port and bind address
Database	MYSQLDATABASE, MYSQLUSER, MYSQLPASSWORD, SPRING_DATASOURCE_URL	MySQL connection and credentials
Security	ADMIN_PASSWORD, COOKIE_SECURE	Administrative seed behavior and cookie security
CORS	CORS_ALLOWED_ORIGINS	Allowed frontend origins
Operations	DDL_AUTO, HEALTH_SHOW_DETAILS LOG_LEVEL,	Schema policy, logging, health detail visibility
Uploads	UPLOAD_DIR	Persistent event-image storage path

9.3 Container Deployment

The multi-stage Dockerfile is designed so that the final runtime contains only the Java application and runtime dependencies, not the full Node or Maven toolchains. The final container runs as UID 10001, exposes the configured port, creates an uploads directory, and checks `/actuator/health` as a container healthcheck. Docker Compose provides the MySQL service, network wiring, health-gated dependency startup, and persistent volumes for database data and uploads.

9.4 Firebase Hosting / Hybrid Frontend Delivery

The repository contains `firebase.json` and `.firebaserc` configuration for Firebase Hosting. The hosting target is configured to serve `frontend-redesign/dist` with a catch-all rewrite to `index.html`, making it suitable for SPA routing. Firestore rules and indexes are also present, but the primary CampusConnect relational runtime remains the Spring/MySQL system. The Firebase configuration should therefore be treated as an auxiliary hosting/integration layer rather than as a replacement for the core database.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Documentation and Runtime Version Drift

The repository contains conflicting version statements. `pom.xml` declares Spring Boot 4.1.0 and a Java 21 build target, while the CI workflow uses JDK 21. The README and some earlier design documents mention Java 25 and MySQL 8.4, and `docs/design-report.md` contains a Spring Boot 3.4.1 statement. The current build file and CI workflow are the most reliable indicators of what the automated pipeline actually executes. Future documentation maintenance should align all version claims with these authoritative configuration files.

10.2 Core Schema vs. V5 Migration Drift

`database/schema.sql` describes a four-table core schema, while Flyway V5 adds the `clubs` table, multi-role user fields, event club ownership, and registration check-in/ticket fields. Several design documents still describe the older four-table model. This is a valid architectural evolution, but the documentation package has not been fully synchronized with the latest migration. The present report explicitly records this difference instead of treating the older ER/data dictionary as the complete current runtime model.

10.3 Single-Node In-Memory Rate Limiting

`RateLimitingFilter` stores Bucket4j buckets in a Caffeine in-memory cache. This is simple and effective for a single application instance, but limits are not automatically shared between multiple nodes. The code comment itself identifies a future Redis/distributed Bucket4j approach for multi-instance deployments.

10.4 External Registration Model

CampusConnect records user interest/registration information but may redirect the user to an external registration URL. Therefore the platform is not currently a full internal ticketing/payment system, and local registration data should not be interpreted as a source of financial settlement or final ticket allocation.

10.5 Hybrid Frontend and Transitional Architecture

The repository contains both a React/Vite frontend and legacy Thymeleaf templates. This can be practical during migration, but it introduces multiple presentation paths and requires careful ownership of route behavior. The Docker build now packages the React production bundle into Spring Boot static resources, while the server-rendered templates remain available for selected flows.

10.6 Additional Known Boundaries

- The distributed database documents describe replication, sharding, and outbox evolution, but a multi-node replica/sharding topology is not deployed by the main repository workflow.
- The recommendation engine is intentionally lightweight and linear; it is not a semantic vector-search system.
- Firebase/Firestore rules exist in the repository, but the main application data model remains MySQL-backed.
- The exact total of test cases is not stated consistently across repository documents; this report therefore counts test files and reports CI gate outcomes instead of repeating a conflicting badge total.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Unify and refresh the documentation set against the latest V5 schema and current runtime versions.
- Add explicit internal capacity/ticket allocation if CampusConnect evolves from interest tracking to authoritative ticketing.
- Add time-zone-aware scheduling and recurring calendar support.
- Add notification workflows for event reminders, venue changes, and registration state changes.
- Introduce durable distributed rate limiting for multi-instance deployments.
- Expand the recommendation engine to semantic search/vector retrieval behind a feature flag while retaining lexical fallback.
- Provide stronger privacy, retention, and audit policies for attendee data.

11.2 Possible Integrations or Optimizations

Enhancement	Current State	Possible Evolution
Distributed cache / rate limit	Caffeine in-process cache	Redis-backed shared Bucket4j state
Messaging	Outbox table present	Kafka/RabbitMQ or managed message bus consumer
Search	SQL title/venue/category queries	Hybrid lexical + vector search with authorization-aware filtering
Scalability	Modular monolith	Extract bounded contexts only when deployment/team ownership justifies it
Observability	Actuator + Prometheus + structured logging	Centralized traces, logs and long-term metrics retention
Ticketing	External registration redirect	Optional internal ticket issuance, capacity reservation and payment integration

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

CampusConnect is a database-centered university event platform that combines a modern React frontend with a Spring Boot application and MySQL relational database. Its architecture separates presentation, security, business services, persistence, and operational concerns while keeping the system deployable as a modular monolith. The platform provides student discovery, administrative event management, organizer workspaces, attendance check-in, recommendations, live telemetry, and a transactional outbox primitive for reliable asynchronous event propagation.

12.2 What Was Achieved

- Implemented a relational event and registration model with integrity constraints and indexes.
- Implemented role-aware student, organizer, and administrator workflows.
- Added club-scoped organizer functionality and event-specific access checks.
- Added concurrency protection through pessimistic locking and a unique registration invariant.
- Added explainable recommendations and real-time SSE telemetry.
- Added Flyway migrations, Docker packaging, health/metrics endpoints, and automated GitHub Actions verification.
- Maintained a substantial evidence package covering normalization, query optimization, transactions, distributed design, and recovery/operations planning.

12.3 Skills Learned

The project provides practical experience in relational modeling, normalization, indexing, SQL, ORM design, transaction boundaries, concurrency control, secure web application design, REST API construction, React application architecture, automated testing, containerization, CI workflows, observability, and release-oriented documentation. Equally important, the repository demonstrates the need to distinguish implemented behavior from aspirational architecture and to keep documentation synchronized with the actual build and migration state.

CHAPTER 13: REFERENCES

- [1] **CampusConnect repository** - <https://github.com/tejaswin-amara/campus-connect>
- [2] **CampusConnect README** - README.md in repository
- [3] **CampusConnect project structure** - PROJECT_STRUCTURE.md in repository
- [4] **CampusConnect technical guide** - TECHNICAL_GUIDE.md in repository
- [5] **Database schema** - database/schema.sql and `src/main/resources/db/migration/`
- [6] **ER model** - database/er-diagram.md
- [7] **Requirements** - docs/requirements.md
- [8] **Concurrency evidence** - docs/concurrency-test.md
- [9] **Query optimization evidence** - docs/query-optimization.md
- [10] **Distributed database design** - docs/distributed-database.md
- [11] **Event lifecycle** - docs/event-lifecycle.md
- [12] **Spring Boot documentation** - <https://spring.io/projects/spring-boot>
- [13] **Spring Security documentation** - <https://spring.io/projects/spring-security>
- [14] **Spring Data JPA documentation** - <https://spring.io/projects/spring-data-jpa>
- [15] **MySQL Reference Manual** - <https://dev.mysql.com/doc/>
- [16] **Flyway documentation** - <https://documentation.red-gate.com/flyway/>
- [17] **React documentation** - <https://react.dev/>
- [18] **Vite documentation** - <https://vite.dev/>
- [19] **Docker documentation** - <https://docs.docker.com/>
- [20] **GitHub Actions documentation** - <https://docs.github.com/actions>
- [21] **Firebase Hosting documentation** - <https://firebase.google.com/docs/hosting>
- [22] **OpenAPI / Swagger** - <https://swagger.io/specification/>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

A.1 - Concurrency-safe event lookup using a pessimistic write lock (`EventRepository.java`)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT e FROM Event e WHERE e.id = :id")
Optional<Event> findByIdForUpdate(@Param("id")Long id);
```

A.2 - Database invariant that prevents duplicate user-event registration (`database/schema.sql`)

```
CONSTRAINT uk_user_event UNIQUE (user_id, event_id)
```

A.3 - Real-time telemetry endpoint (`EventTelemetryController.java`)

```
@GetMapping(value = "/api/events/{id}/telemetry-stream",
produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter streamEventTelemetry(@PathVariable Long id,
@RequestParam(defaultValue = "false") boolean probe) {
return telemetryService.subscribeCapacity(id);
}
```

A.4 - Explainable recommendation signal (`RecommendationService.java`)

```
if (categoryWeight > 0) {
score += categoryWeight * 6;
reasons.add("Based on your interest in " + event.getCategory() + " events");
}
if (fillRate > 0.8d) {
score += 8;
reasons.add("Popular choice: seats are filling up");
}
```

Appendix B: Installation / Setup Instructions

```
# 1. Configure the environment
cp .env.example .env

# 2. Start the local stack
docker compose up --build -d

# 3. Verify the backend
./mvnw -B verify

# 4. Run the Spring application directly (optional)
./mvnw spring boot:run

# 5. Frontend-only development (optional)
cd frontend-redesign
pnpm install
pnpm run dev
```

Exact ports and credentials are environment-dependent. The current application.properties defaults the application to port 8080; CI and historical README examples may use other development ports.

Appendix C: Demo Credentials Note

The repository contains local/demo credentials in its README, demo script, seed scripts, and tests. Because some of those artifacts use different demonstration identities and passwords, this report does not treat any single credential pair as a universal production truth. For evaluation, use the credentials documented by the current local setup/seed path and never reuse them in a public deployment. Rotate all demo credentials before exposing the application to an external audience.

Appendix D: API Reference Summary

Area	Representative Endpoints	Purpose
Public events	GET `/api/events`; GET `/api/events/{id}`	Catalogue and event detail
Registration	POST `/api/events/{id}/register`	Record registration/interest state
Organizer	GET/POST `/api/organizer/events`; GET `/api/organizer/events/{id}/attendees`	Club-scoped organizer workspace
Check-in	POST `/api/organizer/events/{id}/check-in`	Verify attendee and record attendance
Telemetry	GET `/api/events/{id}/telemetry-stream`; GET `/api/organizer/events/{id}/live-checkin-stream`	Server-Sent Events
Operations	GET `/actuator/health`; GET `/actuator/prometheus`	Health and metrics
Documentation	GET `/v3/api-docs`; GET `/swagger-ui.html`	OpenAPI and Swagger UI

Appendix E: Repository Visuals

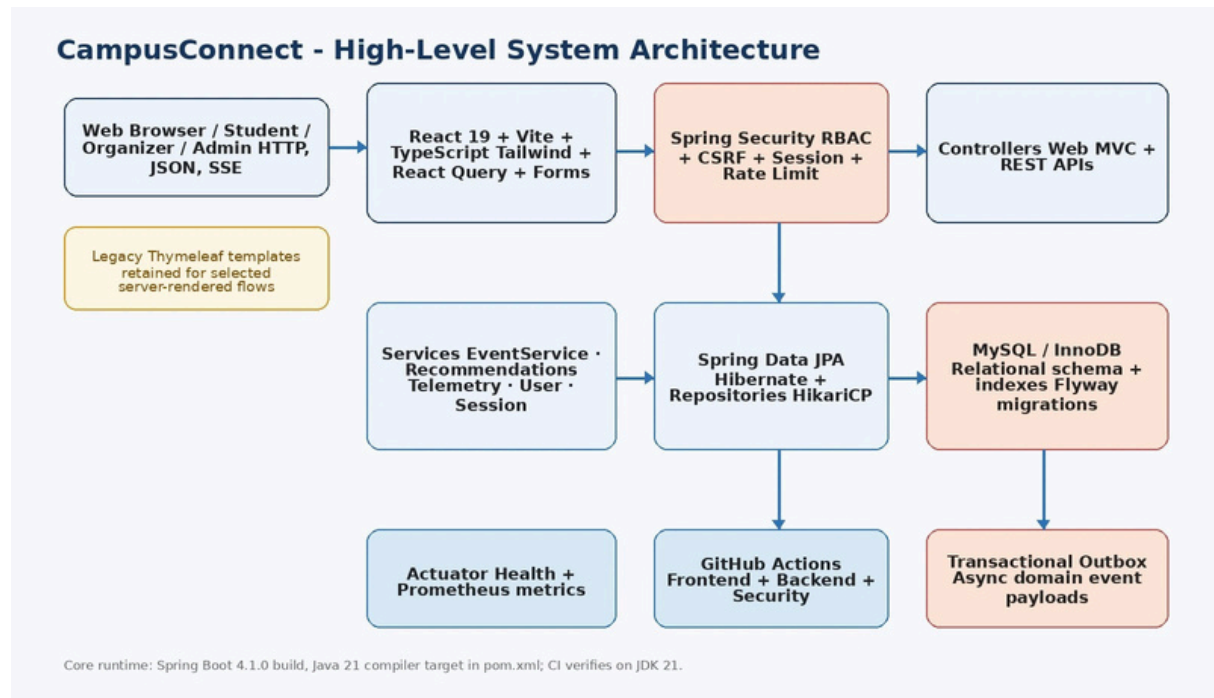
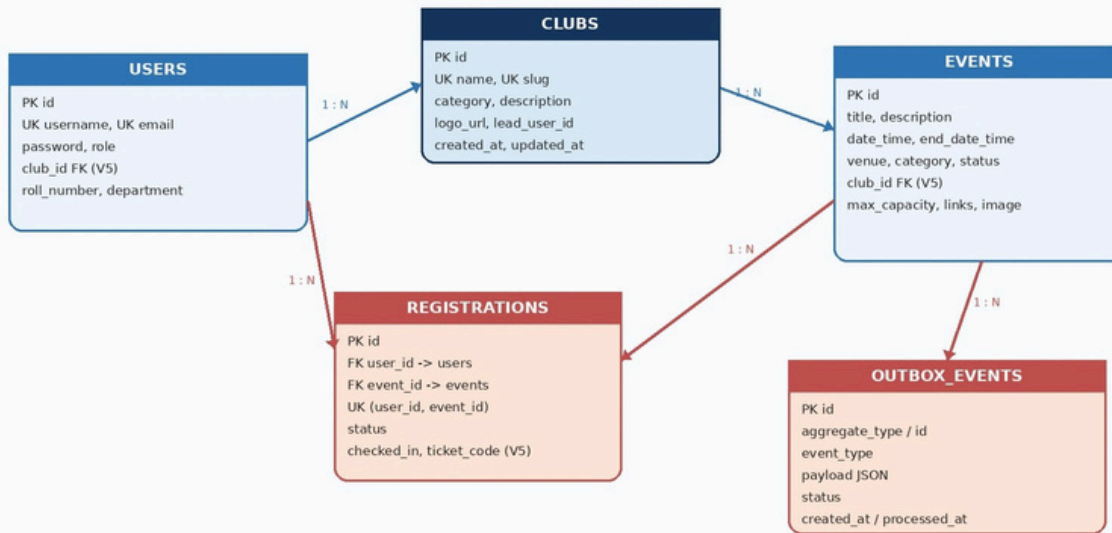


Figure E.1 - Architecture diagram

CampusConnect - ER Model (Current Runtime Scope)



Important repository note: database/schema.sql documents a four-table core schema; Flyway V5 adds clubs, role/club fields, and check-in/ticket telemetry. The report treats V5 as the current runtime extension.

Figure E.2 - Current ER model

CampusConnect - Event Lifecycle

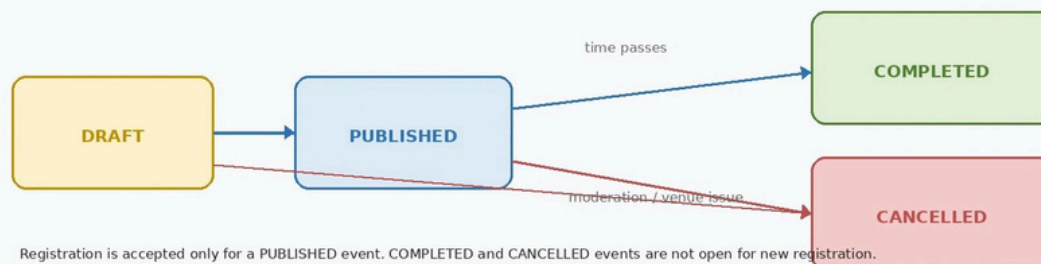


Figure E.3 - Event lifecycle

The repository also contains UI captures under `presentation\_assets/`, including the student catalogue, admin dashboard, create-event form, admin login, event detail view, system architecture, and ER diagram.

Appendix F: Verification Snapshot

Item	Current Value
Main branch	main
Latest reviewed commit	b51fd90e8321c432d3f654945aa2dfef3a60aeb1
Latest reviewed commit date	29 September 2026
CI run	Production Delivery Pipeline - run 87; Frontend, Backend and Security jobs: Success
Repository inventory	21 backend Java test classes; 19 frontend test/spec files; 89 frontend
Flyway migrations	TS/TSX files V1 through V5

Verification note: values above describe repository structure and the latest GitHub Actions run inspected for this report. They are not a substitute for executing a fresh local build against the exact same environment.